

Big Fam Festival — Deep Analysis: Hardening, Optimization & Gaps (Re-Run)

Date: 2026-08-09 (re-run against latest `dev`) **Scope:** Full-stack review — NestJS backend, Firebase Cloud Functions, React Native (Expo) mobile app, Vite/React admin panel, Terraform/GCP infrastructure, CI/CD, and Firestore/Storage security rules. **Method:** Read-only static analysis with file/line evidence, verifying each finding against current file contents. No code was modified. **Branch:** `analysis/deep-dive-hardening` (even with `origin/dev @ dfbee05`)

Executive Summary

This is a **re-run** of the deep analysis against the freshly pulled `dev` branch. Big Fam Festival remains a **production-grade, Firebase-centric festival platform** with a solid engineering foundation. The re-verification confirmed that **most Critical launch-blockers are still present**, surfaced a few **new findings**, and recorded **genuine improvements** plus **one correction** to the prior report.

What changed since the last run

Area	Prior finding	Current state	Delta
Monitoring/alerting (Terraform)	❌ None	<code>infrastructure/terraform/monitoring.tf</code> adds alert policy (5% error rate) + dashboard	🟢 Improved (notification channels still empty)
Firestore composite indexes	❌ Largely missing	Some composite indexes present in <code>firestore.indexes.json</code>	🟢 Partially improved (still missing <code>users.email</code> , <code>friendRequests.*</code> , <code>users.nameLowercase</code>)
Push token registration	❌ No endpoint	<code>PUT /users/push-token</code> + <code>updatePushToken()</code> now exist	🟢 Improved (FCM role + single-token + no mobile caller remain)
Backend admin authorization	Reported as "client-side only"	Backend does enforce <code>@Roles(Role.ADMIN)</code> server-side	✅ Correction — dual-gated; only the Vite UI gate is client-side (UX)
Mobile token persistence	"tokens in AsyncStorage"	ID tokens are not persisted; cached PII profiles are (AsyncStorage, plaintext)	✅ Nuance/correction
Firestore users/notifications rules	🔴 World-open	Still allow read, write: if true / allow read: if true	❌ Unchanged
Hardcoded mobile Firebase keys	🔴 Committed	Still hardcoded in <code>app.config.js</code> + <code>google-services.json</code> + <code>plist</code>	❌ Unchanged

Cloud Run public + scale	🔴 allUsers, min0/max10	Unchanged	❌ Unchanged
N+1 / unbounded scans	🔴 Present	Unchanged (+ new getFriendCampsites/Locations 1+2N found)	❌ Unchanged / worse detail
CI security scanning	❌ None	Still no SAST/dep/secret/container scanning; ci-gate.yml still a placeholder	❌ Unchanged
Terraform state hardening	🟡 Unencrypted	Unchanged	❌ Unchanged

Top risks at a glance

#	Risk	Severity	Area
1	Firestore users + notifications collections world-readable/writable	🔴 Critical	Data / Rules
2	Hardcoded production Firebase API keys committed in mobile repo	🔴 Critical	Secrets
3	N+1 / unbounded Firestore scans (friends.search, sendRequest, getFriendCampsites/Locations, admin.listUsers/getStats)	🔴 Critical	Performance
4	Cloud Run publicly invocable (allUsers), min=0/max=10 instances	🔴 Critical	Infra / Scale
5	No request body size limit + CORS wildcard default	🟡 High	Backend Security
6	No SAST / dependency / secret / container scanning in CI; ci-gate placeholder	🟡 High	Supply Chain
7	Public @Public() debug endpoints leak token count / FCM config	🟡 High	Info Disclosure
8	Push delivery still unreliable (FCM role, single-token, no mobile caller)	🟡 High	Reliability
9	No transactions/batching → race conditions on friend requests & schedules	🟡 High	Data Integrity
10	Mobile: plaintext PII cache, no cert pinning, no deep-link allowlist, Sentry disabled in boundary	🟡 High	Mobile Security

Bottom line: Production is **not ready for a live event**. The Critical items must be remediated first. Estimated blocking remediation: ~2–3 weeks of focused work.

1. Backend (NestJS + Cloud Functions)

Stack: NestJS 10 + TypeScript 5.1, Node 18+ (Docker) / Node 22 (Functions), Firestore + Cloud Storage, Firebase Admin SDK 13.4. Security: Helmet 8, @nestjs/throttler . Validation: class-validator + Joi (env). Logging: nestjs-pino . Email: SendGrid. Push: expo-server-sdk .

1.1 Security

- **Permissive Firestore rules** — firestore.rules:9 allow read, write: if true on /users/{userId} exposes email, phone, profile, and location-sharing prefs to unauthenticated clients (in-file TODO acknowledges it, deferred from PR #133). Default catch-all allow read, write: if request.auth != null remains over-permissive.
- **CORS wildcard** — main.ts:33 defaults CORS_ORIGIN='*' with credentials: true ; .env.production.example:3 ships CORS_ORIGIN=* . Cross-origin CSRF for authenticated sessions.
- **No request body size limit** — main.ts (~L45) applies compression() / helmet() but never sets express.json({ limit }) ; relies on implicit 100 KB default → DoS via large payloads.
- **@Public() debug endpoints** — backend/src/debug/notifications-debug.controller.ts exposes GET /debug/notifications/token-count , GET /debug/notifications/fcm-config , and a POST test endpoint **unauthenticated** → user-count enumeration, FCM config disclosure, push-spam abuse.
- **Weak TypeScript config** — tsconfig.json disables strictNullChecks , noImplicitAny , strictBindCallApply , noFallthroughCasesInSwitch .
- **Unbounded search input** — friends.controller.ts q param has no @MaxLength ; combined with full-scan search, a DoS lever.
- **console.warn bypasses pino** — e.g. users.service.ts:82 logs migration failures outside structured logging (no correlation ID, no redaction).
- **Good:** Global ValidationPipe (whitelist + forbidNonWhitelisted); Firebase Auth guard verifies revocation (verifyIdToken(token, true)) with new-account fallback; **admin endpoints are server-side gated** via @Roles(Role.ADMIN) on admin.controller.ts (corrects prior report); Helmet applied; prod error redaction in http-exception.filter.ts .

1.2 Performance

- **N+1 / full-collection scans (still present):**
 - friends.service.ts:24 searchUsers() loads **all** users then filters in JS.
 - friends.service.ts:60 sendRequest() fetches all pending + accepted requests and dedups in memory.
 - friends.service.ts:285 getFriendCampsites() and :333 getFriendLocations() are **1 + 2N** read patterns (per-friend user doc + campsite/location doc). 100 friends ≈ 201 reads.
 - admin.service.ts:44 listUsers() loads all users, filters/sorts/paginates in memory (TODO: "move filtering to Firestore query when user count exceeds 1K").
 - admin.service.ts:23 getStats() reads entire users , events , notifications collections per dashboard load.
- **Missing Firestore indexes** — firestore.indexes.json has users(userGroups, expoPushToken) and notifications(sentAt) , but still lacks users.email , users.nameLowercase , users.role , and friendRequests(fromUserId,toUserId,status) / (toUserId,status) / (fromUserId,status) . The listUsers TODO cannot be executed without these.
- **No caching layer** — user profiles, events, and static genres re-read every request.

- **No retry/backoff** — `firestore.service.ts` awaits directly; transient `UNAVAILABLE / DEADLINE_EXCEEDED / RESOURCE_EXHAUSTED` surface as hard failures.
- **No transactions/batching** — single-doc ops only → friend-request create/response and schedule reset are non-atomic (race conditions, duplicates).
- **Cloud Functions partial-failure** — `functions/src/index.ts` (~L168) catches per-chunk push errors and continues without retry; best-effort delivery.

1.3 Reliability

- **Push delivery still unreliable** — token registration endpoint now exists, but: FCM service account still hits `FCM permission denied` (needs `cloudmessaging.admin`); single-token-per-user overwrites multi-device; **no mobile code calls the registration endpoint** (no `PUT /users/push-token` caller found in the app).
- **No graceful shutdown** — no `SIGTERM` in-flight drain; Cloud Run force-kills after 10s.
- **No idempotency keys** on create operations (friend requests, notifications).
- **No correlation-ID propagation** downstream (pino has `req.id` locally only).
- ✓ **Good:** `/health` + `/health/ready`; startup Firestore connection test (non-fatal); structured pino logs with auth-header redaction.

1.4 Testing

- ~7 backend spec files (events, auth, guards, filters, map, festival-day) + a single health e2e; **no service tests** for `friends / admin / users`; no coverage threshold enforced in Jest config. k6 load suite (4K VUs) exists but is **not wired into CI** and has no tracked baseline.

2. Mobile App (React Native / Expo)

Stack: Expo 54, React Native 0.81.5, React 19.1, React Query 5, React Navigation 6, `@rnmapbox/maps` 10, `@react-native-firebase` 23, Sentry 7, Axios 1.12, TypeScript 5 **strict**.

2.1 Security

- **Hardcoded production Firebase credentials (still committed):**
 - `app.config.js:26` — `apiKey: 'AIzaSyDxZIs1o0TetHu0SsuV300f84RTCDkmg0s'`, project/sender/app IDs hardcoded (dev creds are env-gated; **prod is unconditionally hardcoded**).
 - `google-services.json` (Android) + `GoogleService-Info.plist` (iOS) committed with real config.
 - **Action:** rotate all keys in GCP, move to EAS/Expo secrets, scrub git history, add `google-services*.json / GoogleService-Info*.plist` to `.gitignore`, rebuild + redeploy.
- **Plaintext PII cache** — `AuthContext.tsx:91` writes `cachedUserProfile` (userId, role, email, phone, ticket type) to `AsyncStorage` (unencrypted). *Nuance:* Firebase **ID tokens are not persisted** (re-fetched via `getIdToken()`), which is good — the exposure is the cached profile, not the token. Move sensitive fields to `expo-secure-store`.
- **No certificate pinning** — `src/services/api.ts:53` plain Axios; MITM-exposed (Cloud Run, Mapbox, Firebase, Sentry).
- **No deep-link validation** — `EventDetailsModal.tsx:304` builds `big-fam://map?stage=...` and calls `Linking.openURL()` with no scheme/domain allowlist; push-driven deep links planned without validation.
- **Sensitive logging not fully __DEV__ -gated** — `api.ts:68` logs API URLs/requests; `friendService.ts:70` logs search queries with usernames. (`.eslintrc.js` `no-console` allows `warn / error` only, so `log` should be gated.)

- 🟡 **Hardcoded Google OAuth client IDs** — `firebaseAuthService.ts:21` (public-by-design, lower priority).
- 🟡 **No jailbreak/root/integrity attestation; no unhandled-promise-rejection handler.**
- ✅ **Good:** `usesCleartextTraffic: false` ; scoped Android permissions (ACTIVITY_RECOGNITION blocked); error boundary; offline queue; NetInfo detection; Axios exponential backoff on 5xx/429; Mapbox camera throttling; `React.memo` ; **TypeScript strict**.

2.2 Performance / Quality

- 🟡 **Hermes not enabled** — `app.json` lacks `jsEngine: "hermes"` → ~30% slower Android startup + larger bundle.
- 🟡 **Sentry `captureException` commented out** in `ErrorBoundary.tsx:50-55` — React crashes not reported (Sentry is otherwise initialized and filters auth headers).
- 🟡 **React Query `staleTime` flat 5 min** (`App.tsx:40`) — reasonable, but static POI/schedule data could use longer per-domain values.
- 🟡 **Test coverage** — only ~5 test files, 50% threshold, no Detox/E2E.
- ✅ **Good (verified):** `ScheduleScreen` `FlatList` is well-optimized — memoized `keyExtractor` , `maxToRenderPerBatch: 20` , `updateCellsBatchingPeriod: 100` , image pre-fetch + lookahead, `gs://` → `https://` handling.

3. Admin Panel (Vite + React)

Stack: React 18, React Router 6, Vite 5, Radix UI + Tailwind, React Query 5, Mapbox GL 3.

- ✅ **Authorization is dual-gated (correction):** the Vite `useAuth.tsx` gate is UX-only, but the **backend re-verifies** every privileged call via `@Roles(Role.ADMIN)` . Not a privilege-escalation hole as previously implied.
- 🟡 **No security headers** in `vite.config.ts` (no CSP / X-Frame-Options / X-Content-Type-Options).
- 🟡 **No audit logging** of admin CRUD actions.
- ✅ No hardcoded secrets found; Firebase-managed auth; RBAC routes gated by `RequireAdmin` .

4. Infrastructure (Terraform / GCP)

- 🔴 **Cloud Run publicly invokable** — `cloud-run.tf:83` binds `roles/run.invoker` to `allUsers` ; `min_instances=0` , `max_instances=10` . Public abuse surface + cold starts + ~500 req/s ceiling (a 4K-attendee gate rush needs ~1,500–2,000 req/s).
- 🟡 **Terraform state risk** — `main.tf` GCS backend `bigfam-terraform-state` shows no encryption key, versioning, or locking → secret leakage + concurrent-apply races.
- 🟡 **IAM breadth** — Cloud Run SA holds `roles/datastore.user` + `roles/storage.objectAdmin` (can delete all objects); tighten to least privilege / specific buckets.
- 🟡 **No GCP Secret Manager** — secrets passed via env vars only (SendGrid key, etc.).
- 🟢 **Monitoring/alerting ADDED** — `monitoring.tf` defines a high-error-rate alert policy (5% > 60s) + a request/error dashboard. **Gap:** `notification_channels = []` (no PagerDuty/email wired), so alerts fire into the void.
- 🟡 **No disaster recovery** — no automated Firestore exports, no RTO/RPO, no region-failover plan.

5. CI/CD & Repo Hygiene

Workflows: `backend-ci` (lint/typecheck/test/build/deploy), `mobile-ci` (lint/test/EAS build), `android-deploy` (strict, build-ID-pinned alpha deploy), `release-please`, plus advisory gates.

- 🟡 **No SAST / dependency / secret / container scanning** anywhere in CI (grep found none).
Hardcoded Firebase keys demonstrably slipped through as a result.
- 🟡 **ci-gate.yml is a placeholder** that only `echo s "CI gate passed"` — no real quality gate.
- 🟡 **No coverage enforcement** in CI.
- 🟡 **.gitignore** correctly ignores `backend/bigfamfestival-*.json`, `backend/.env*`, `mobile/.env*` — but `*.tfstate` isn't explicit, and committed `app.config.js` prod keys aren't caught.
- 🟡 **Pre-commit hook not auto-enabled** — requires manual `git config core.hooksPath .githubhooks`.
- ✅ **Good:** `android-deploy` pins exact EAS build ID + verifies commit SHA, profile, and expiry — strong release discipline.

6. Firestore / Storage Rules Summary

Rule	Location	Verdict
users allow read, write: if true	firestore.rules:9	🔴 Critical — PII exposed
notifications allow read: if true	firestore.rules:38–40	🔴 Critical — all notifications public
Default allow read, write: if request.auth != null	firestore.rules:44	🟡 High — over-permissive catch-all
campsites / userLocations owner-only	firestore.rules	✅ Correct
Storage admin writes via token.role == 'admin' + default deny	storage.rules	✅ Correct (role claim verified server-side)
@Public() debug notification endpoints	backend/src/debug/notifications-debug.controller.ts	🟡 High — info disclosure

7. Gaps the Team May Not Be Thinking About

1. **Event-scale capacity planning** — Firestore's 1 write/sec/doc limit will bottleneck shared docs when hundreds update simultaneously; needs **sharded writes** and a true-concurrency load test before gates open. Cloud Run max=10 won't absorb a gate rush.
2. **Disaster recovery & data retention** — No automated backups, no RTO/RPO, no GDPR/CCPA right-to-be-forgotten flow for attendee PII and **real-time location** data (warrants a DPIA + explicit consent UI).
3. **Observability last mile** — Monitoring/dashboards now exist, but **no notification channels are wired**, no SLOs, and no on-call runbook for a live multi-day event.
4. **Abuse prevention** — Global 100 req/60s only; no per-user/per-IP limits, no CAPTCHA on auth, no login-velocity/brute-force protection, no email/phone verification gate. Public Cloud Run + open debug endpoints widen this.

5. **Session management for multi-day events** — Short-lived JWT with no refresh-rotation endpoint means mid-festival logouts; no session revocation on compromise.
 6. **Multi-tenant / white-label isolation** — Terraform supports `festival_id`, but Firestore rules don't scope by it; a shared project risks cross-tenant leakage.
 7. **API versioning strategy** — `/api/v1` exists but there's no documented deprecation/dual-version policy for shipping breaking changes to installed apps.
 8. **Supply-chain provenance** — Docker images pushed unscanned; Artifact Registry write scope unreviewed.
-

8. Prioritized Remediation Roadmap

Phase 0 — Launch Blockers (Weeks 1–3)

1. Tighten Firestore rules: owner-scoped `users`, recipient-scoped `notifications`, remove permissive default. **(1)**
2. Rotate committed Firebase keys; move mobile prod secrets to EAS env; scrub git history; gitignore `google-services*` /plist. **(2)**
3. Convert in-memory scans to indexed Firestore queries + add composite indexes (`users.email / nameLowercase / role`, `friendRequests(fromUserId,toUserId,status)` etc.); denormalize campsite/location onto `friends` subcollection to kill the 1+2N reads. **(3)**
4. Restrict Cloud Run invoker (drop `allUsers` or front with API gateway/Cloud Armor); set `minScale≥2`, raise `maxScale` from load-test results. **(4)**
5. Add body size limit + CORS allowlist; gate/remove `@Public()` debug endpoints. **(5, 7)**
6. Add SAST + `npm audit` + secret scanning + container scanning to CI; replace placeholder `ci-gate`. **(6)**
7. Fix push delivery: grant FCM `cloudmessaging.admin`, store token arrays, and wire the mobile app to call `PUT /users/push-token`. **(8)**
8. Encrypt + version + lock Terraform state.

Phase 1 — Pre-Festival Hardening (Weeks 3–5)

- Introduce Firestore transactions/batching for friend requests & schedule writes. **(9)**
- Mobile: `expo-secure-store` for cached PII, certificate pinning, deep-link allowlist, wire up Sentry in the error boundary, gate remaining `console.log` behind `__DEV__`, add unhandled-rejection handler. **(10)**
- Backend: retry/backoff, graceful shutdown, correlation-ID propagation, `@MaxLength` on search input.
- Wire monitoring **notification channels**; define SLOs; author an incident runbook; automate daily Firestore backups.
- Add per-user/per-IP rate limits, CAPTCHA + email verification on auth.

Phase 2 — Sustaining Quality (Post-launch)

- Raise backend + mobile coverage to 80%; wire k6 + Detox into CI with tracked baselines.
 - Enable `tsconfig` strict flags (backend); enable Hermes (mobile).
 - Add admin audit logging + Vite security headers; caching layer (LRU); token refresh rotation + revocation.
 - Document API versioning/deprecation policy; add GDPR/CCPA + DPIA docs and consent UI.
 - Enforce `festival_id` scoping in rules for white-label isolation; adopt GCP Secret Manager.
-

Appendix — Key Evidence Index

Finding	File	Line
Users collection world-open	firestore.rules	9
Notifications public read	firestore.rules	38–40
Over-permissive default rule	firestore.rules	44
Friends N+1 scan	backend/src/friends/friends.service.ts	24–43
sendRequest unbounded query	backend/src/friends/friends.service.ts	60–83
getFriendCampsites 1+2N	backend/src/friends/friends.service.ts	285–323
getFriendLocations 1+2N	backend/src/friends/friends.service.ts	333–368
Admin listUsers in-memory	backend/src/admin/admin.service.ts	44–83
Admin getStats full reads	backend/src/admin/admin.service.ts	23–38
Server-side admin RBAC (good)	backend/src/admin/admin.controller.ts	32–34
Public debug endpoints	backend/src/debug/notifications-debug.controller.ts	7–35
CORS wildcard	backend/src/main.ts	33–39
Missing body-size limit	backend/src/main.ts	45–47
Weak TS config	backend/tsconfig.json	9–13
Push token endpoint (new)	backend/src/users/users.controller.ts	73
FCM permission error	backend/src/notifications/notifications.service.ts	409
Hardcoded prod Firebase key	mobile/app.config.js	26–37
Plaintext PII cache	mobile/src/contexts/AuthContext.tsx	91
No cert pinning	mobile/src/services/api.ts	53
Deep link unvalidated	mobile/src/components/EventDetailsModal.tsx	304
Sentry disabled in boundary	mobile/src/components/ErrorBoundary.tsx	50–55
Cloud Run public + scale	infrastructure/terraform/cloud-run.tf	83–102
Terraform state unhardened	infrastructure/terraform/main.tf	1–12
Monitoring added (channels empty)	infrastructure/terraform/monitoring.tf	—
CI gate placeholder	.github/workflows/ci-gate.yml	—

Generated from a read-only multi-agent re-analysis on 2026-08-09 against `origin/dev` @ `dfbee05`.
Line references reflect the state of the `dev` branch at analysis time.